Як організувати роботу з LLM над великими технічними задачами

Практичний підхід до рефакторингу, аудиту кодової бази, масових перейменувань і міграцій без хаотичних правок від AI

Зміст

Як організувати роботу з LLM над великими технічними задачами: методологія Analysis -> Plan -> Execution **3**
1. Головна ідея: не змішувати аналіз, план і виконання 3
2. Phase 0 — Context Loading: спочатку завантажити контекст 4
3. Phase 1 — Analysis: побудувати повну картину порушень 5
4. Phase 2 — Task Map: перетворити аналіз на формальну карту задач 6
5. Phase 3 — Execution Plan: перетворити карту на порядок роботи 9
6. Phase 4 — Execution: змінювати код контрольованими проходами 9
7. Phase 5 — Validation: підтвердити, що роботу справді виконано 11
8. Як застосовувати методологію на практиці: набір промптів 12
9. Методологія не обмежується перейменуванням CSS-класів 14
10. Що робити з файлами на сотні й тисячі рядків 14
11. Критерії завершення: як вимірювати результат 15
12. Типові помилки, від яких захищає підхід 15
Висновок 16

Як організувати роботу з LLM над великими технічними задачами: методологія Analysis -> Plan -> Execution

Практичний підхід до рефакторингу, аудиту кодової бази, масових перейменувань і міграцій без хаотичних правок від AI.

LLM уже вміють читати код, знаходити підозрілі конструкції та пропонувати зміни. Але на складних задачах проблема часто не в тому, що модель не знає синтаксису. Проблема в організації роботи: вона починає виправляти першу знайдену помилку, ще не розібравшись у всьому обсязі робіт; пропускає залежності; плутає вимоги; змінює кілька файлів одночасно; а перевірку відкладає до кінця.

Результат знайомий багатьом розробникам: частина правок правильна, частина неповна, деякі зміни конфліктують між собою, а перевірити, чи виконано всі вимоги, складно.

`AI Task Execution Methodology` пропонує інший підхід: **не просити модель одночасно аналізувати, планувати й змінювати код**. Роботу потрібно розділити на послідовні етапи, перетворити результати аналізу на структуровану карту задач, виконувати зміни контрольованими порціями та перевіряти результат після кожного файлу.

Методологія описана у Markdown-файлі `ai-task-execution-methodology.md`. Це не програмний агент і не готовий скрипт автоматизації. Це робочий протокол, за яким розробник організовує взаємодію з LLM.

1. Головна ідея: не змішувати аналіз, план і виконання

Основний принцип методології можна записати так:

```
ANALYSIS -> PLAN -> EXECUTION
```

Це три різні типи роботи.

- **Analysis** — з'ясувати, що саме не відповідає вимогам, де розташовані проблеми та від чого вони залежать.
- **Plan** — зафіксувати знайдені проблеми у формалізованому вигляді й визначити безпечний порядок змін.
- **Execution** — виконати погоджені зміни та перевірити результат.

У робочому процесі ці три етапи деталізовані до шести фаз: від `Phase 0 — Context Loading` до `Phase 5 — Validation`.

Чому не варто виправляти проблему одразу після її виявлення

Припустімо, під час рефакторингу модель знаходить CSS-змінну `--msg-bubble-radius`, яку за правилами проєкту потрібно перейменувати на `--messages-bubble-radius`.

Наївний сценарій простий: модель змінює назву змінної, переходить до наступного порушення й так далі. Але стара назва може використовуватися в інших CSS-правилах, у JavaScript-коді, у тестах або в конфігурації. Якщо модель не простежила всі використання, код після перейменування може частково перестати працювати.

Методологія вимагає спочатку з'ясувати **повний обсяг проблеми**, потім записати основну зміну та всі залежні зміни, а вже тоді виконувати їх як один логічний блок.

Тут важливо розрізняти два поняття:

- **Порушення** — конкретне відхилення від специфікації.
- **Domino dependency** — інші місця, які потрібно змінити разом із цим порушенням, щоб не залишити систему в частково оновленому стані.

У прикладі з CSS-змінною основна проблема — неправильна назва токена. Її domino-залежностями можуть бути `var(--msg-bubble-radius)` у CSS і `getPropertyValue('--msg-bubble-radius')` у JavaScript.

Отже, одиницею планування стає не просто рядок коду, а **зміна разом із її залежностями**.

2. Phase 0 — Context Loading: спочатку завантажити контекст

Перший етап не передбачає ані аналізу проблем, ані виправлення коду. Його завдання — переконатися, що модель отримала всі матеріали, які визначають межі роботи.

Методологія виділяє три групи контексту:

1. **Specification / guide** — специфікація, стандарт або гайд, якому має відповідати результат.
2. **Materials** — файли, код, дані та інші робочі матеріали.
3. **Constraints** — обмеження: які файли не можна змінювати, що є критичним, які пріоритети та винятки потрібно зберегти.

На цьому етапі LLM має тільки прочитати матеріали та підтвердити, що завантаження завершене.

Як це виглядає в чаті

Спочатку окремим повідомленням передаємо гайд:

> Прочитай повністю `containerization-6.md`. Нічого не аналізуй і не змінюй. Після читання підтвердь, що файл прочитано повністю, та коротко назви його призначення.

Після цього передаємо робочі файли:

> Прочитай усі файли, перелічені нижче. На цьому етапі нічого не виправляй і не пропонуй змін. Після читання поверни таблицю: файл, кількість рядків, призначення, чи вдалося прочитати його повністю.

Навіщо розділяти ці повідомлення

Коли гайд, файли, аналіз і команда на редагування потрапляють в один запит, моделі легше перейти до виконання, не зафіксувавши, які саме правила є обов'язковими. Окрема фаза знижує ризик того, що робота почнеться на неповному або неправильно зрозумілому контексті.

Практичне обмеження: підтвердження моделі саме по собі не гарантує, що вона справді бачила всі байти всіх файлів. Якщо інструмент обрізає вкладення, не відкриває частину файлів або обмежує контекст, це потрібно виявити й вирішити до аналізу. Для великої кодової бази корисно передавати явний список файлів і перевіряти його повноту.

3. Phase 1 — Analysis: побудувати повну картину порушень

Аналіз у методології — це не загальне прохання «перевір код». Він має бути систематичним і виконуватися за визначеними категоріями.

3.1. Спочатку створити таксономію порушень

Таксономія — це явний перелік типів проблем, які потрібно знайти. Для рефакторингу назв у методології наведено, наприклад, такі категорії:

Категорія	Що перевіряємо
`CSS_PREFIX`	Префікси та скорочення в CSS-класах або токенах
`KEYFRAME_PREFIX`	Назви CSS-анімацій у `@keyframes`
`DATA_CUSTOM`	Користувацькі HTML-атрибути без необхідного префікса
`DATA_ACTION_VALUE`	Формат значень у `data-action`
`ID_PREFIX`	Відповідність `id` правилам іменування віджета
`ID_DEAD`	`id`, які оголошені в HTML, але ніде не використовуються
`ID_JS_HOOK`	Використання `id` як JavaScript-гачків усупереч архітектурному правилу
`JS_VAR`	Скорочення та невідповідні назви JavaScript-змінних

Це приклади для конкретного сценарію, а не універсальний список для всіх проєктів. Для аудиту безпеки, міграції API або перевірки доступності потрібні інші категорії.

Ключовий момент: **категорії мають покривати весь узгоджений обсяг перевірки**. Якщо забути окремий клас порушень, модель може якісно знайти проблеми в межах наданого списку, але все одно пропустити проблеми іншого типу.

3.2. Аналізувати за схемою «файл × категорія»

Після формування таксономії модель послідовно перевіряє кожен файл за кожною категорією. Не варто дозволяти їй перескакувати між файлами та виправляти проблеми на ходу.

Для рефакторингу можна використати такий порядок контрольних питань:

1. Що порушено в CSS?
2. Що порушено в HTML?
3. Що порушено в JavaScript?
4. Які `id` використовуються, а які є мертвими?
5. Які порушення мають залежності в інших місцях?

Цей порядок можна адаптувати, але він має бути явним. Мета — зробити аналіз повторюваним і перевірюваним, а не залежним від того, на що модель звернула увагу першою.

3.3. Виявляти domino dependencies

Для кожного порушення потрібно відповісти на запитання: **що ще перестане працювати або залишиться неконсистентним, якщо виправити лише це місце?**

Наприклад, для перейменування CSS-токена слід перевірити:

- оголошення змінної в `:root` або іншому CSS-селекторі;
- усі використання через `var(...)`;
- читання токена з JavaScript через `getPropertyValue(...)`;
- тести, документацію або інші файли, якщо вони входять до узгодженого scope.

Важлива вимога — залежності повинні бути конкретними. Формулювання «оновити JS» надто нечітке: незрозуміло, де саме і що змінювати. Краще вказати файл, місце, старий фрагмент і пов'язаний ідентифікатор проблеми.

Якщо domino-аналіз неповний, рефакторинг може завершитися станом, у якому частина коду вже використовує нову назву, а інша частина все ще очікує стару.

4. Phase 2 — Task Map: перетворити аналіз на формальну карту задач

Результат аналізу — не просто список зауважень у прозі. Методологія пропонує створювати **Task Map** — структурований артефакт, у якому кожна проблема має ідентифікатор, категорію, серйозність, точне місце, очікувану зміну, залежності та статус.

Для карти використовується JSON. Це дозволяє зберігати її окремо від діалогу, автоматично перевіряти її структуру, посилатися на конкретні проблеми та відстежувати прогрес.

4.1. Які поля потрібні для проблеми

Поле	Призначення
`id`	Унікальний ідентифікатор проблеми для посилань із залежностей
`category`	Код категорії порушення
`severity`	Рівень серйозності, який впливає на порядок виконання
`change_type`	Тип операції: перейменування, видалення, заміна тощо
`location.lines`	Точний діапазон рядків у файлі на момент аналізу
`location.context`	Дослівний фрагмент коду, за яким можна знайти місце
`location.scope`	Контекст розташування: наприклад, `CSS:tokens`
`current`	Поточне значення або конструкція
`expected`	Очікуване значення або конструкція
`description`	Коротке пояснення порушення
`rule_ref`	Посилання на конкретний пункт стандарту
`dependencies`	Перелік domino-залежностей із посиланнями на інші `issue_id`
`status`	Стан виконання: `pending`, `in-progress` або `done`

Мініатюрний приклад карти:

```
{
  "meta": {
    "total_issues": 2,
    "guide_version": "containerization-6"
```

```
  },
  "files": [
    {
      "file": "widget.html",
      "issues": [
        {
          "id": "widget-001",
          "category": "CSS_PREFIX",
          "severity": "HIGH",
          "change_type": "RENAME",
          "location": {
            "lines": { "start": 69, "end": 82 },
            "context": "--msg-bubble-radius: 0.75rem;",
            "scope": "CSS:tokens"
          },
          "current": "--msg-bubble-radius",
          "expected": "--messages-bubble-radius",
          "description": "Скорочення msg не відповідає стандарту.",
          "rule_ref": "§7.1",
          "dependencies": [
            {
              "issue_id": "widget-002",
              "file": "widget.html",
              "description": "Використання токена у var(...)"
            }
          ],
          "status": "pending"
        },
        {
          "id": "widget-002",
          "category": "CSS_PREFIX",
          "severity": "HIGH",
          "change_type": "RENAME",
          "location": {
            "lines": { "start": 165, "end": 166 },
            "context": "var(--msg-bubble-radius)",
            "scope": "CSS:rule"
          },
          "current": "--msg-bubble-radius",
          "expected": "--messages-bubble-radius",
          "description": "Посилання на перейменований токен.",
          "rule_ref": "§7.1",
          "dependencies": [],
          "status": "pending"
        }
      ]
    }
  ]
}
```

Приклад скорочений для пояснення структури. У реальній карті потрібно врахувати всі знайдені проблеми, а не лише головний фрагмент.

4.2. Чому точного тексту й номерів рядків недостатньо окремо

`location.lines` потрібні, щоб швидко знайти місце. `location.context` потрібен, щоб розпізнати його, якщо після попередніх правок рядки змістилися.

Номери рядків — це координати на момент створення карти, а не незмінна адреса. Після видалення або вставлення коду вони можуть стати неактуальними. Тому варто зберігати обидва типи інформації й під час виконання звірятися з реальним файлом.

Контекстний фрагмент має бути дослівним і достатньо коротким для однозначного пошуку. У методології як орієнтир наведено обмеження до 120 символів.

4.3. Severity: що виправляти раніше

Методологія пропонує чотири рівні серйозності:

- **CRITICAL** — код зламаний або порушено архітектурний інваріант. Прикладом у файлі є використання `id` для пошуку DOM-вузла з JavaScript, якщо локальний стандарт забороняє такий підхід.
- **HIGH** — порушення правил іменування або інша проблема, що створює значний ризик для масштабування й узгодженості.
- **MEDIUM** — проблема, яка не змінює поведінку, але створює зайвий шум, наприклад мертвий `id`.
- **LOW** — стилістичне відхилення.

Ця шкала — інструмент пріоритизації, а не універсальна класифікація ризику для будь-якого продукту. Її визначення потрібно адаптувати до задачі.

4.4. Change type: що саме робити з кодом

Для чіткості операції мають власні типи:

- `RENAME` — змінити назву, зберігши функцію;
- `DELETE` — повністю видалити конструкцію;
- `REPLACE` — замінити одну конструкцію іншою;
- `ADD` — додати відсутній елемент;
- `MOVE` — перемістити елемент в інше місце.

Різниця між, наприклад, `RENAME` і `DELETE` важлива. Перше передбачає збереження сутності та оновлення її використань, друге — перевірку, що сутність справді не потрібна.

4.5. Перевіряти саму карту

Карта теж може містити помилки. Тому перед виконанням варто перевірити щонайменше таке:

- усі `id` унікальні;
- кожен `issue_id` у `dependencies` існує в карті;
- усі проблеми мають категорію, серйозність, місце й тип зміни;
- `current` і `expected` не змішані в одному текстовому описі;

- кількість у `meta.total_issues` відповідає реальному переліку;
- scope чітко описує, що включено в роботу, а що не потрібно змінювати.

Зв'язок через `issue_id` кращий за довільний текст на кшталт «змінити ще місця в JS», оскільки його можна перевірити автоматично.

5. Phase 3 — Execution Plan: перетворити карту на порядок роботи

Task Map відповідає на питання «що не так?». План виконання відповідає на питання «у якому порядку це виправляти?». Це не одне й те саме.

Методологія пропонує групувати план у три рівні:

1. **Файл** — верхній рівень групування.
2. **Тип порушення** — підгрупа всередині файлу.
3. **Конкретна проблема** — окремий пункт зі статусом.

Умовний приклад:

```
widget.html
├── CSS prefix
│   ├── [ ] widget-001 — перейменувати токен (domino: 2)
│   └── [ ] widget-004 — перейменувати клас (domino: 5)
├── @keyframes
│   └── [ ] widget-009 — виправити назву анімації
└── HTML id
    └── [ ] widget-013 — перевірити та перейменувати id
```

План може бути звичайним Markdown-чеклістом, JSON, завданнями у трекері або іншим зрозумілим артефактом. У методології вимога стосується насамперед структури й порядку, а не конкретного UI.

Пріоритети виконання

Базова послідовність така:

1. `CRITICAL` — архітектурні порушення;
2. `HIGH` — іменування та інші зміни з великим ризиком залежностей;
3. `MEDIUM` — мертвий код і подібне очищення;
4. `LOW` — стилістичні правки.

У межах одного рівня серйозності методологія пропонує раніше виконувати проблеми з більшою кількістю domino-залежностей. Логіка проста: чим більше місць пов'язано зі зміною, тим важливіше розібрати її контрольовано, перш ніж накопичувати інші правки.

Це правило варто застосовувати разом із реальними обмеженнями проєкту. Наприклад, якщо певна зміна потребує міграції API або зупинки сумісності, її планування може вимагати окремого релізного етапу.

6. Phase 4 — Execution: змінювати код контрольованими проходами

На цьому етапі модель нарешті редагує файли. Але вона не отримує свободу «покращити все, що бачить». Виконання прив'язане до карти й плану.

6.1. Один файл за раз

Файл відкривають, виконують усі заплановані для нього зміни, перевіряють результат і тільки після цього переходять до наступного.

Це зменшує кількість одночасних змінних у роботі. Якщо після редагування з'явилася помилка, простіше визначити, у якому файлі та на якому етапі вона виникла.

6.2. Виконувати domino-залежності разом із головною зміною

Якщо `widget-001` має три залежності, не варто перейменовувати тільки головне оголошення, залишаючи решту на потім. Основна зміна й усі залежності, необхідні для збереження узгодженості, виконуються як один блок.

Після завершення потрібно оновити статуси відповідних пунктів карти. Не можна позначати залежність як `done`, якщо її місце фактично не перевірено.

6.3. Для складного файла — кілька логічних проходів

Методологія пропонує обробляти складні файли пошарово. Наприклад:

```
Pass 1: CSS — токени, класи, keyframes
Pass 2: HTML — атрибути
Pass 3: JS — рядки та вирази
Pass 4: id — видалення мертвих і перейменування живих
```

Порядок конкретних проходів залежить від типу задачі. Принцип полягає в тому, щоб групувати зміни за логічним шаром, а не робити довільні заміни по всьому файлу.

Це особливо корисно, коли використовуються масові текстові заміни або регулярні вирази: кілька незалежних правил заміни можуть конфліктувати, якщо застосувати їх у неправильній послідовності.

6.4. Спочатку найдовші та найспецифічніші шаблони

Якщо загальна заміна виконується першою, вона може знищити фрагмент, за яким мав спрацювати конкретніший шаблон.

Невдалий порядок:

```
replacements = [
    ('.msg-msg', '.messages-bubble'),
    ('.msg-msg--sent', '.messages-bubble--sent'),
]
```

Після заміни `.msg-msg` рядок `.msg-msg--sent` уже може не збігтися з вихідним шаблоном.

Безпечніший порядок — спершу конкретніші й довші варіанти:

```
replacements = [
    ('.msg-msg--sent', '.messages-bubble--sent'),
    ('.msg-msg--recv', '.messages-bubble--received'),
    ('.msg-msg', '.messages-bubble'),
]
```

Цей приклад ілюструє порядок текстових замін. Для складного коду звичайний `str.replace()` не завжди достатній: він може зачепити коментар, текст користувача, іншу назву або фрагмент, який не є ідентифікатором. У таких випадках краще використовувати контекстні пошукові шаблони, парсери чи інструменти рефакторингу, а потім перевіряти diff.

6.5. Не відкладати перевірку до завершення всіх файлів

Правило методології: **файл -> перевірка -> наступний файл**.

Якщо обробити вісім файлів і лише потім запускати пошук залишкових шаблонів, буде складніше локалізувати джерело проблеми. Перевірка відразу після кожного файла дає коротший цикл зворотного зв'язку.

7. Phase 5 — Validation: підтвердити, що роботу справді виконано

Факт, що модель повідомила «готово», не є перевіркою. Методологія розділяє валідацію на кілька рівнів.

7.1. Автоматична перевірка шаблонів

Після перейменування варто пошукати старі назви, які мали зникнути. Для невеликого прикладу це можна зробити регулярним виразом у Python:

```
import re

remaining = [
    line
    for line in src.splitlines()
    if re.search(r'\bmsg-', line)
    and not line.strip().startswith(('#', '//', '<!--', '*'))
]
```

Цей фрагмент — лише ілюстрація принципу. Він не є універсальним валідатором CSS, HTML чи JavaScript: правила початку коментаря залежать від мови, багаторядкові коментарі потребують окремої обробки, а збіг `msg-` може бути легальним у сторонньому контексті.

Для реальної перевірки потрібно адаптувати пошук до форматів файлів і правил конкретного проєкту. Якщо є відповідні статичні аналізатори, лінтери або AST-інструменти, їхні результати часто надійніші за один загальний regex.

7.2. Семантична перевірка винятків

Не кожен збіг пошукового шаблону є порушенням. Тому після автоматичного пошуку потрібно переглянути залишкові входження та зрозуміти їхній контекст.

Наприклад, у специфікації певного віджета може бути дозволено шукати контейнер через клас:

```
document.querySelector('.widget-container'); // допустимо за цим правилом
```

Водночас пошук вузла через `id` може порушувати локальний архітектурний стандарт:

```
document.querySelector('#some-id'); // порушення, якщо так заборонено в гайді
```

Це не універсальна заборона для всього вебу. Важливо застосовувати саме правила проєктного гайда, а не перетворювати приклад із методології на загальну догму.

Так само `id` може бути потрібним для зв'язку HTML-елементів через ARIA-атрибути. Тому його не можна видаляти лише тому, що JavaScript не знаходить його в пошуку: потрібно перевірити всі передбачені способи використання.

7.3. Зіставити карту з результатом

Коли виконання завершено, перевіряють:

- чи відповідає кількість закритих пунктів карті;
- чи немає старих шаблонів, які мали зникнути;
- чи всі файли з переліку оброблено;
- чи не змінено семантику там, де план вимагав лише перейменування;
- чи залишилися легальні винятки й чи вони пояснені;
- чи пройшли потрібні тести, збірка або smoke test.

Завершальний звіт за файлами має показувати статус і кількість залишкових проблем. Якщо частина пунктів не виконана або не перевірена, це потрібно вказати прямо, а не оголошувати весь рефакторинг завершеним.

8. Як застосовувати методологію на практиці: набір промптів

Нижче — приклад послідовності запитів, яку можна адаптувати до Cursor, IDE-асистента або звичайного чату з доступом до файлів. Конкретні можливості читання й редагування залежать від інструмента.

Крок 1. Завантажити гайд

```
Прочитай повністю файл [шлях до гайда].
Нічого не аналізуй, не змінюй і не пропонуй правок.
Після читання підтвердь завершення та коротко опиши призначення документа.
Якщо файл недоступний або прочитаний не повністю — прямо повідом про це.
```

Крок 2. Завантажити матеріали та зафіксувати scope

```
Прочитай усі файли зі списку нижче.
Не змінюй їх і не починай аналіз.
Поверни таблицю: шлях, кількість рядків, призначення, результат читання.
Не додавай до scope файли, яких немає у списку.
Якщо якийсь файл недоступний або його вміст обрізаний, назви його окремо.
```

Крок 3. Побудувати категорії й виконати аналіз

```
Працюй лише в режимі аналізу. Не редагуй файли.
Перевір кожен файл зі scope за кожною категорією порушень:
[вставити повний перелік категорій і їхні визначення].
Для кожного порушення зафіксуй файл, точні рядки, дослівний контекст,
поточне й очікуване значення, посилання на правило та рівень серйозності.
Для кожної проблеми знайди всі відомі domino-залежності в межах scope.
Якщо впевнено підтвердити залежність неможливо, познач невизначеність,
не вигадуй місце або посилання.
Поверни результати аналізу, але не виправляй їх.
```

Крок 4. Згенерувати Task Map

```
На основі завершеного аналізу створи Task Map у JSON за схемою:
[вставити точну JSON-схему].
У кожної проблеми мають бути унікальний id, category, severity,
change_type, location.lines, location.context, current, expected,
rule_ref, dependencies та status.
Кожна залежність має посилатися на issue_id, що реально існує в карті.
Не додавай проблем, яких немає в результатах аналізу.
Після генерації перевір JSON на синтаксис, унікальність id,
наявність усіх dependency reference та відповідність total_issues.
На цьому етапі код не змінюй.
```

Крок 5. Побудувати план

```
Перетвори Task Map на план виконання.
Групування: файл -> категорія порушення -> окремий issue.
Спочатку CRITICAL, потім HIGH, MEDIUM і LOW.
У межах одного рівня врахуй кількість domino-залежностей.
Для кожного файла визнач послідовні логічні проходи.
Не редагуй код. Поверни план і перелік пунктів, які потребують уточнення.
```

Крок 6. Виконати зміни для одного файла

```
Виконай тільки заплановані зміни у файлі [шлях].
Не переходь до інших файлів.
Для кожної основної проблеми виконай усі її domino-залежності в тому самому
логічному проході. Дотримуйся порядку замін від специфічних до загальних.
Не внось поліпшень, яких немає в карті.
Після редагування покажи перелік виконаних issue_id, фактично змінені місця
та будь-які пункти, які не вдалося виконати. Не позначай неперевірені пункти done.
```

Крок 7. Перевірити файл

```
Перевір змінений файл за всіма пунктами його Task Map.
Виконай пошук старих шаблонів, перевір domino-залежності,
```

```
розглянь залишкові збіги на предмет легальних винятків і зістав diff
з початковим scope. За можливості запусти відповідні лінтери й тести.
Поверни таблицю: issue_id, очікуваний результат, фактичний результат,
статус, пояснення. Лише після перевірки онови статуси.
Після цього зупинись — до наступного файла не переходь.
```

Наведені промпти — шаблони, а не магічні команди, що гарантують повноту. Якість результату залежить від точності специфікації, доступності файлів, якості інструментів пошуку та перевірки, а також від того, чи справді розробник переглядає diff і підтверджує критичні зміни.

9. Методологія не обмежується перейменуванням CSS-класів

У Markdown-файлі наведено кілька типових сценаріїв, для яких змінюються категорії аналізу та правила планування.

Codebase Audit

Для аудиту кодової бази перевірка може охоплювати безпеку, продуктивність, доступність і правила іменування. У карті, крім серйозності, можна передбачити оцінку зусиль. План у цьому сценарії групують за виміром аудиту, а після виконання запускають регресійні тести.

API Migration

Для міграції API спочатку завантажують документацію старої та нової версій, а потім знаходять усі використання застарілих методів. Breaking changes позначають як критичні й за потреби ізолюють в окремі партії. Після кожного файла проводять тести, а наприкінці шукають старі шаблони й виконують smoke test.

Large-Scale Rename

Для масового перейменування бренду, простору імен або префікса потрібно шукати всі входження не лише в реалізації, а й у тестах, коментарях і документації — відповідно до scope. План може передбачати порядок: типи й інтерфейси, реалізація, тести, документація. Наприкінці виконують фінальний пошук, збірку та тести.

Ці сценарії демонструють головний принцип: **сталі фази можна повторно використовувати, але правила перевірки й критерії завершення повинні відповідати конкретному типу задачі**.

10. Що робити з файлами на сотні й тисячі рядків

Для файлів понад 500 рядків методологія радить отримати повне уявлення про файл до початку змін, а не читати зверху вниз і редагувати одразу після кожного знайденого фрагмента.

Також краще збирати однотипні правки в пакети. Наприклад, усі CSS-токени можна описати в одному словнику старих і нових назв, а потім виконати окремий CSS-прохід. Після кожного проходу перевіряють, чи залишились старі назви, перш ніж переходити до HTML або JavaScript.

Однак повне читання — це вимога до процедури, а не припущення, що будь-яка модель без обмежень утримує в контексті необмежену кількість коду. Якщо інструмент не може надійно прочитати весь файл, потрібно використовувати засоби індексації, пошуку й поетапного завантаження, зберігаючи карту задач між кроками. Не можна мовчки прирівнювати частковий огляд до повного аналізу.

Для пакетних замін важливо враховувати, що прості заміни рядків не розуміють семантику мови. Вони можуть змінити коментарі, текст інтерфейсу, сторонні назви або фрагменти, що лише випадково збігаються з шаблоном. Для складних випадків застосовуйте мовні інструменти, AST-рефакторинг, лінтери й перевірку diff.

11. Критерії завершення: як вимірювати результат

Методологія рекомендує порівнювати стан до і після роботи та фіксувати числові показники:

- кількість файлів у scope;
- загальну кількість рядків;
- кількість проблем за рівнями серйозності;
- кількість файлів без порушень.

Окремо перевіряють, чи дорівнює число закритих проблем кількості пунктів карти, чи зникли всі шаблони, що мали бути видалені, чи відповідають `id` і `data-action` правилам конкретного гайда, а також чи збереглася семантика коду там, де мали змінитися лише назви.

Кількісні показники не доводять відсутності всіх помилок, але допомагають зробити прогрес вимірюваним і виявити очевидну невідповідність між заявленим та фактичним обсягом роботи.

Для продукційного коду ці перевірки варто доповнювати стандартним інженерним процесом: version control, невеликі diff, тести, збірка, статичний аналіз і можливість відкотити зміни. Методологія організовує роботу LLM, але не замінює тестування, code review або відповідальність розробника за результат.

12. Типові помилки, від яких захищає підхід

Аналіз і виправлення в одному кроці. У процесі губиться загальна картина, а модель починає виконувати зміни, не знаючи повного переліку робіт.

Залежності описані нечітко. Текст «оновити JS» не можна надійно перевірити. Зв’язок за `issue_id` дає змогу підтвердити, що всі залежні зміни існують у карті.

Неправильний порядок масових замін. Загальний шаблон може змінити рядок раніше, ніж спрацює специфічний. Правило — конкретніші та довші шаблони першими.

Валідація лише наприкінці. Помилка, внесена на початку, може бути виявлена значно пізніше. Перевірка після кожного файла локалізує джерело проблеми.

Сліпа довіра до regex. Пошук знаходить збіги, але не визначає їхню легальність. Кожен залишковий збіг треба інтерпретувати в контексті.

Карта без точного місця. Формулювання «у методі обробки подій» не дає достатньої основи для відтворюваної перевірки. Потрібні рядки й дослівний контекст.

Невизначені межі роботи. Якщо не вказати, що саме входить у scope і що не можна змінювати, модель може розширити задачу власними припущеннями.

Висновок

`AI Task Execution Methodology` перетворює взаємодію з LLM із серії спонтанних команд на контрольований інженерний процес. Спочатку модель отримує повний контекст, потім аналізує всі файли за визначеними категоріями, фіксує проблеми та їхні залежності в Task Map, будує план, виконує зміни невеликими логічними порціями й перевіряє результат після кожного файла.

Основна цінність — не в самому JSON і не в переліку промптів. Вона в розділенні відповідальностей між етапами та в тому, що результат кожного етапу стає вхідними даними для наступного. У такий спосіб легше виявити пропуски до редагування, зрозуміти взаємозв'язки, локалізувати помилки та зіставити фактичний результат із початковою задачею.

За описом автора, методологію було розроблено під час рефакторингу дев'яти віджетів загальним обсягом 9 007 рядків: у межах роботи було зафіксовано 84 порушення стандарту `containerization-6`. Цей кейс показує тип задачі, для якої підхід створювався, але сам по собі не є універсальним доказом ефективності для будь-якого проєкту.

Для розробника практична рекомендація проста: **не починайте з команди «виправ усе». Почніть із контексту, критеріїв перевірки та карти роботи — і дозвольте LLM виконувати зміни лише після того, як зрозуміло, що саме потрібно зробити й як ви перевірите результат.**